Innovation University

Faculty of Computers and Information Technology (FCIT)

CF223: Computer Networks
CF229: Database Management Systems

Level 2 - Spring Semester

May 2026

InnovAI MEDICAL CENTER

A Next-Generation, **AI-Integrated** Hospital Management System (HMS)

Team Members

ID	Student Name
24030126	*Youssef Sameh Ahmed Noby
24030005	Ahmed Hesham Ramadan Ismail
24030150	Eyad Ayman Mohamed Talaat
24030055	Moatasem Mohamed Mostafa Saied
24030006	Marwan Bahaa Mohammed Elsayed
24030217	Youssef Wael Atef Mohamed

GitHub Repository

https://github.com/yecreate/InnovAI-Hospital-Management-System

Contents

1. Project Overview & Architecture Overhaul

InnovAI Medical Center is a fully functional, next-generation ***Hospital Management System (HMS)*** designed to eliminate operational friction across clinical and administrative environments. Since our initial prototype, the system architecture has undergone a complete overhaul. We have strategically deprecated heavy, client-side browser automation in favor of lightweight, decoupled ***Node.js*** microservices and have upgraded our core intelligence to utilize dynamic, deterministically constrained ***Large Language Models (LLMs)***

The platform operates through five distinct, role-based user portals, delivering seamless patient communication, real-time clinical tracking, and autonomous AI-driven triage routing over a securely segmented network

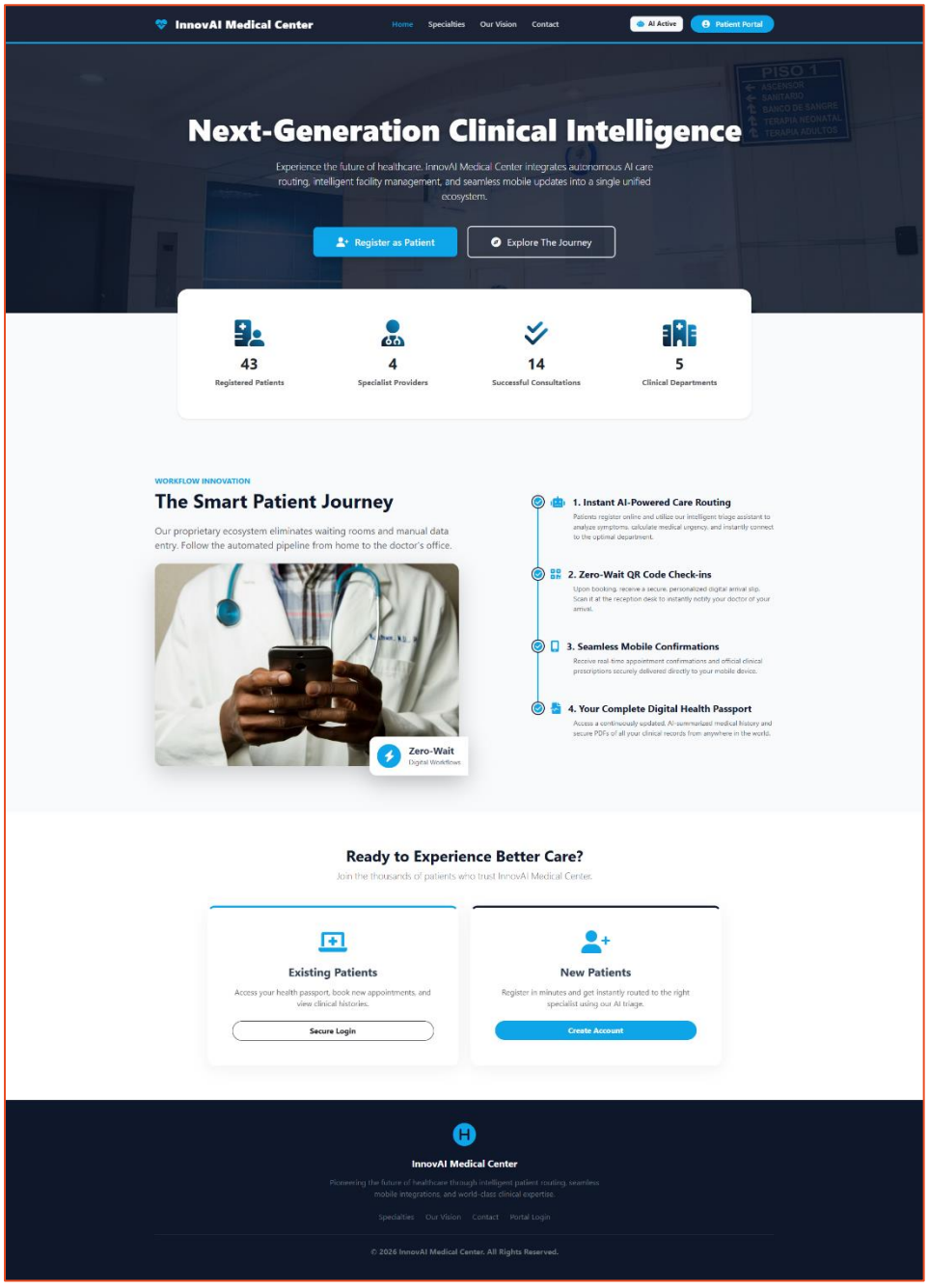

Figure 1 The primary web gateway for InnovAI Medical Center, demonstrating dynamic database-driven statistics and centralized access to the five role-based clinical and administrative portals.

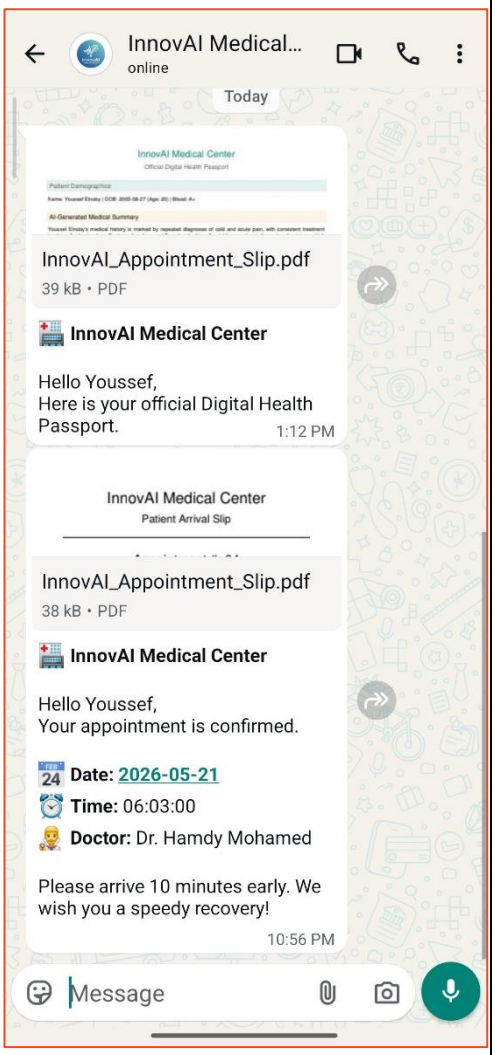

Figure 2 Output from the decoupled Node.js microservice (Port 3000), demonstrating asynchronous delivery of FPDF-generated clinical documents directly to the patient's edge device

2. Database Architecture & Constitutional Rules

The InnovAI database enforces absolute structural integrity, relying on strict relational constraints, automated derived attributes, and rigorous normalization schemas to prevent data anomalies

2.1 The Source of Truth & Active Schema

The database operates on a highly constrained schema consisting of **15 foundational tables**: ***User_Login***, ***Admin***, ***Department***, ***Receptionist***, ***Doctor***, ***Doctor_phones***, ***Nurse***, ***Room***, ***Patient***, ***Patient_phones***, ***Billing***, ***Appointment***, ***Medical_History***, ***Prescription***, and ***AI_Analysis***

Key Schema Waivers & State Logic Additions

- **Operational ENUMS:** The system state logic within the Appointment table is driven by a strict, cyclical ENUM sequence: $ENUM('Scheduled','Checked-In','Completed','Cancelled','Ready')$. The addition of the **Ready** status is a vital, custom protocol enabling the autonomous nurse-to-doctor clinical floor workflow
- **AI Metadata Storage:** The *AI_Analysis* table was granted a constitutional project waiver to map and persist LLM cognitive outputs, adding *Confidence_Level* ($DECIMAL(5,2)$) and Summary_Text to permanently store the generated health passport reasoning logs

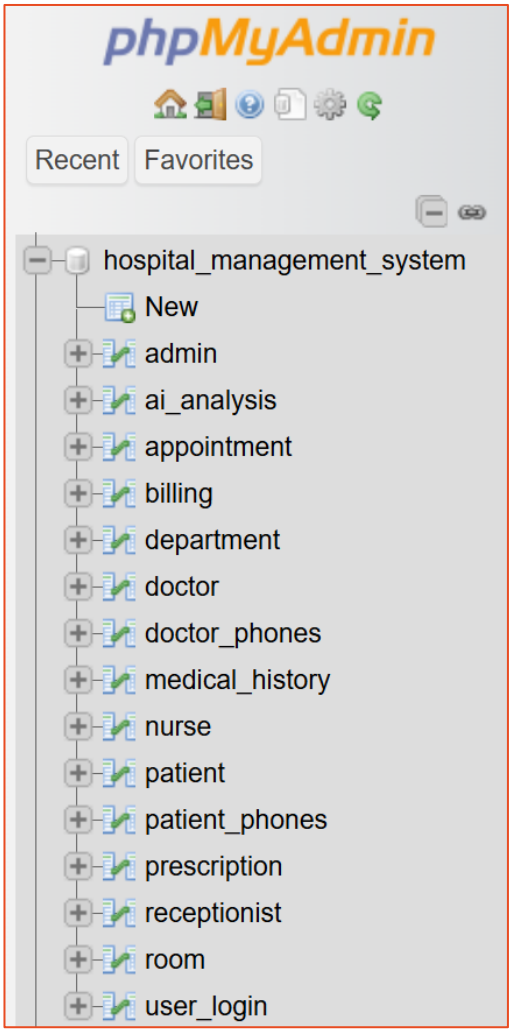

Figure 3 The phpMyAdmin interface displaying the InnovAI database schema, confirming the deployment of the 15 foundational tables.

2.2 Automated Database Triggers

To prevent backend PHP memory overhead and eliminate application-layer interference, physical derived attributes are handled entirely by the MySQL engine. Patient age is calculated and synchronized continuously via asynchronous database triggers hooked directly to the ***Date_of_Birth*** variable:

```
1  CREATE TRIGGER update_patient_age BEFORE INSERT ON Patient;
2  CREATE TRIGGER update patient age on update BEFORE UPDATE ON Patient;
```

Figure 4 Implementation of physical derived attributes via MySQL BEFORE INSERT and BEFORE UPDATE triggers, eliminating backend PHP computation overhead and ensuring data synchronization.

2.3 Relational Normalization Analysis (3NF/BCNF)

The schema was mathematically engineered to satisfy the Boyce-Codd Normal Form (BCNF), ensuring zero insertion, update, or deletion anomalies

2.3.1. Elimination of Multi-Valued Attributes (1NF)

Phone numbers are normalized out of the main entity tables to prevent domain violation. Let $\boldsymbol{R}$ be the conceptual patient relation. We decompose it into two discrete relations:

$$\boldsymbol{Patient} = \{\boldsymbol{Patient\_ID}, \boldsymbol{Fname}, \boldsymbol{Lname}, \boldsymbol{DOB}, \boldsymbol{City}\}$$

$$\boldsymbol{Patient\_phones} = \{\boldsymbol{Patient\_ID}, \boldsymbol{Phone\_Number}\}$$

Here, the primary key of $Patient\_phones$ is the composite $(Patient\_ID, Phone\_Number)$

2.3.2. Boyce-Codd Normal Form (BCNF) Enforcement

For the core entities, every non-trivial functional dependency $X \rightarrow Y$ mandates that X is a superkey. For example, in the Patient table:

$$\boldsymbol{Patient\_ID} \rightarrow \{\boldsymbol{Fname}, \boldsymbol{Lname}, \boldsymbol{DOB}, \boldsymbol{City}\}$$

Because $Patient\_ID$ is the primary key, this dependency inherently satisfies BCNF across the system architecture

---

2.4 Entity-Relationship Mapping & Schema Visualization

To handle the complex routing of a medical facility, the database relies on strict referential integrity mapping across $one-to-one\ (1:1)$, $one-to-many\ (1:M)$, and $many-to-many\ (M:M)$ relationships

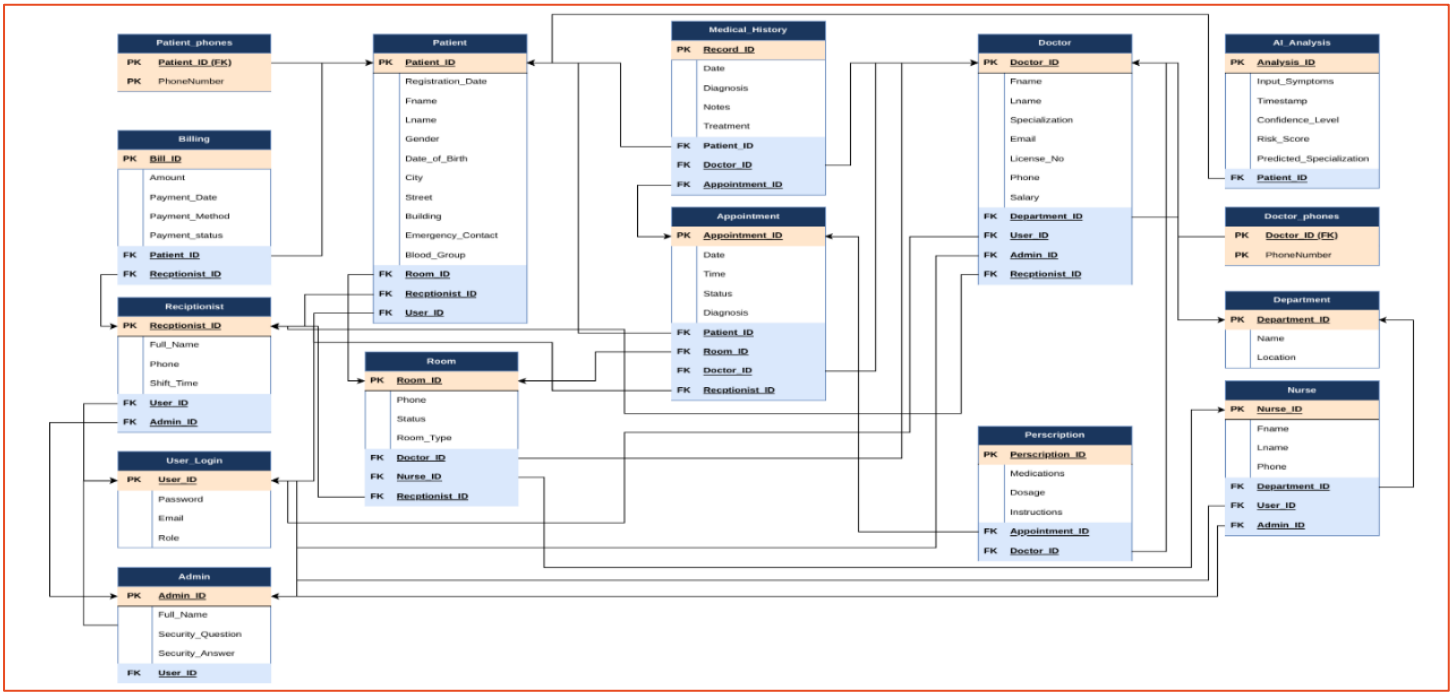

Figure 5 The InnovAI relational schema diagram, detailing the physical table structures, primary keys, and strict foreign key dependencies across the database.

- **Unified Authentication (1:1):** Every role (*Admin*, *Doctor*, *Nurse*, *Receptionist*, *Patient*) maps securely back to the centralized *User_Login* table
- **Cascade Constraints:** Foreign keys are strictly bound with *ON DELETE CASCADE* rules. For instance, removing a patient from the *Patient* table autonomously clears all associated records in the *AI_Analysis*, *Appointment*, and *Patient_phones* tables, eliminating the risk of orphaned data
- **Junction Tables (M:M):** Complex clinical overlaps, such as multiple doctors attending to multiple patients, or receptionists assigning multiple doctors, are resolved flawlessly using dedicated junction tables (*Doctors_Patient* and *Receptionist_Doctors*)

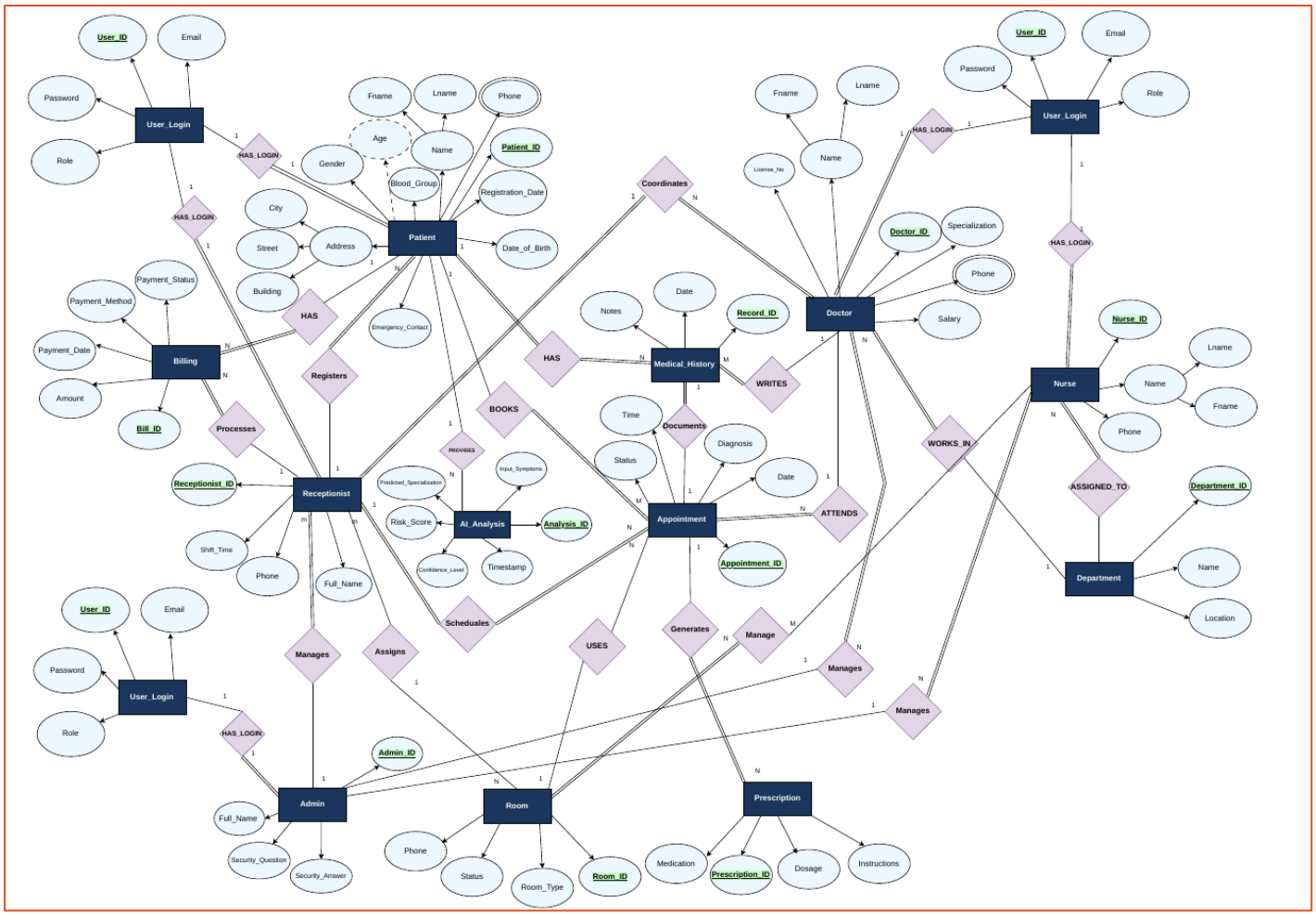

Figure 6 The high-level conceptual ERD demonstrating 1:1, 1:M, and M:M cardinality mappings, including derived physical attributes and isolated multi-valued phone domains.

Ultimately, establishing these rigorous relational blueprints ensures that the database acts as the absolute source of truth. By enforcing relationship constraints and domain integrity at the physical data layer, the system guarantees that the generative AI engine and the asynchronous *Node.js* microservices are exclusively fed sanitized, perfectly normalized data. This architectural rigidity serves as a critical failsafe, preventing downstream runtime errors and guaranteeing zero data anomalies during high-traffic clinical workflows

3. Network Topology & Microservices

The deployment utilizes a decoupled, microservice-based approach running over a simulated segmented network to isolate sensitive clinical data from external communication gateways

3.1 Multi-VLAN Segmented Architecture

To mimic enterprise-grade hospital infrastructure, system traffic is explicitly routed across a logical Multi-VLAN topology:

- **$VLAN\ 10$ (Admin):** Command and control subnet dedicated exclusively to administrative CRUD operations and financial pulse dashboards
- **$VLAN\ 20$ (Clinical):** Highly secure, isolated internal subnet restricted to the Doctor Portal (AI Priority Queue) and Nurse Portal (Live Floor Heatmap)
- **$VLAN\ 30$ (DMZ / Services):** The public-facing demilitarized zone hosting the core XAMPP Web Server and the asynchronous Node.js WhatsApp Microservice gateway

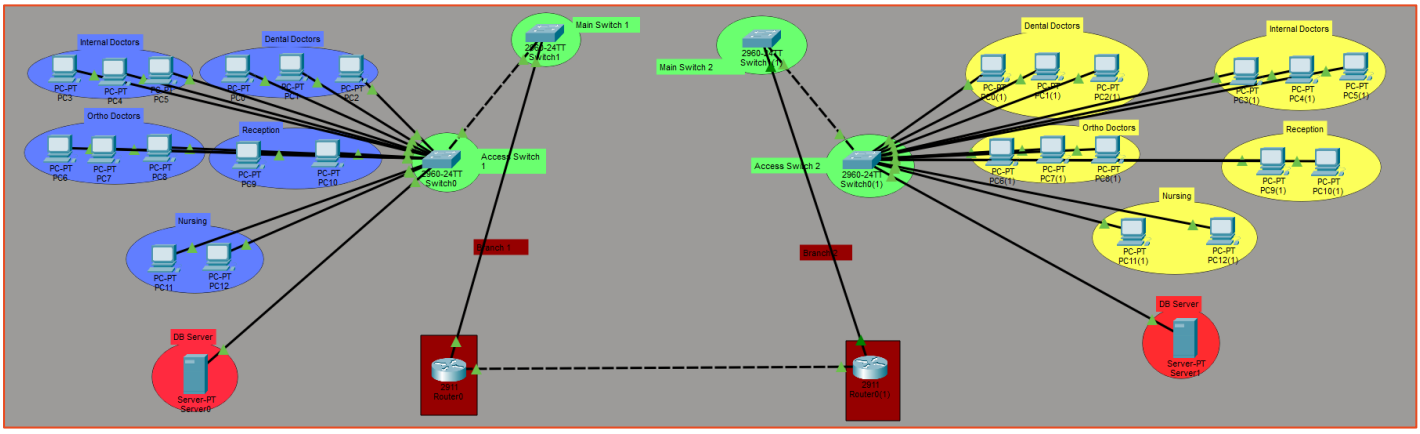

Figure 7 Packet Tracer simulation of the enterprise network architecture, illustrating strict VLAN isolation and physical routing between the clinical subnets and the core database servers.

3.2 Lightweight WebSocket Gateway (Port 3000)

A major architectural triumph of this build was the deprecation of heavy, RAM-intensive headless browsers (Puppeteer) in favor of a direct WebSocket integration.

- **The Microservice:** A bespoke $Node.js$ instance runs on $localhost{:}\ 3000$, completely outside the XAMPP htdocs root. It utilizes the lightweight $@whiskeysockets/baileys$ library paired with a pino logger to interface directly with **WhatsApp's WebSocket servers**
- **Asynchronous Trigger:** The PHP backend acts purely as an API client, utilizing $curl\_init('http://localhost{:}\ 3000/send-msg')$ to fire JSON payloads asynchronously, ensuring the main receptionist UI thread is never blocked during document transmission

4. AI Engine Routing & Clinical Integration

The clinical intelligence core of InnovAI has been upgraded to utilize the $llama-3.3-70b-versatile$ model via the Groq API. To prevent unpredictable outputs, the LLM is operated with absolute determinism by locking the generation variance ($temperature = 0.0$)

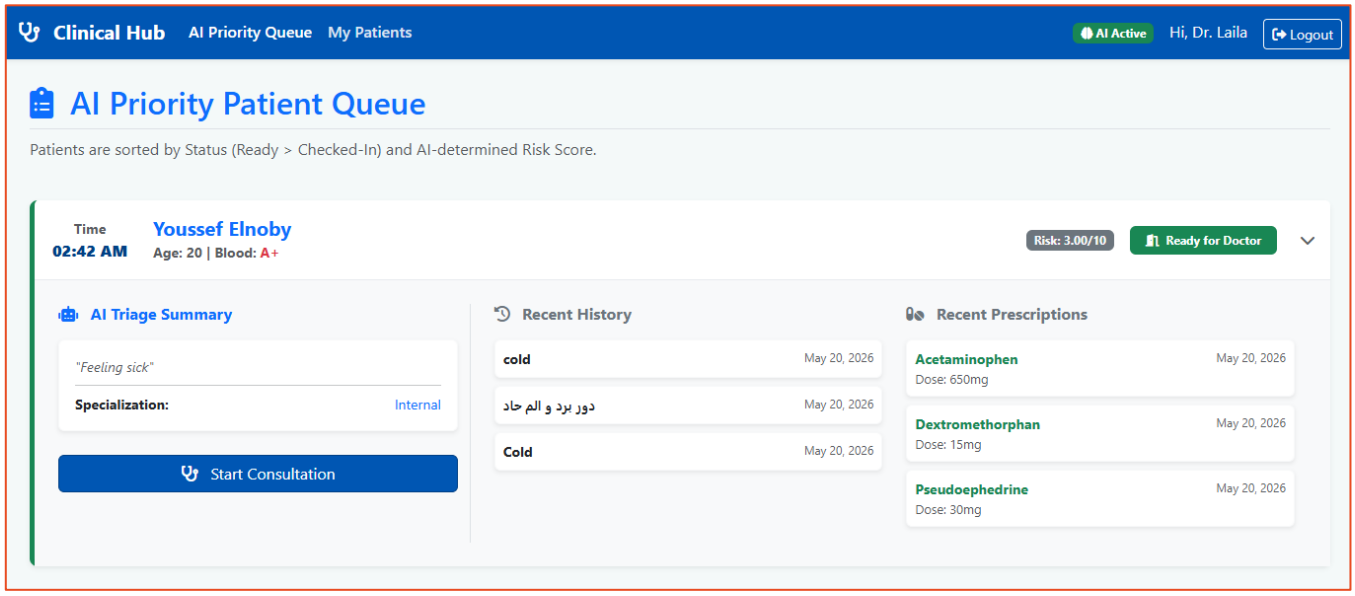

Figure 8 The isolated clinical interface rendering real-time AI triage data, demonstrating the seamless integration of Groq API outputs

4.1 Dynamic Department Fetching & Strict Schema

The system utilizes a dynamic context-injection architecture:

1. Upon a patient launching the Symptom Checker, the PHP backend queries the live database ($SELECT\ Name\ FROM\ Department$)
2. The returned array is imploded into a string and dynamically injected into the LLM's system prompt rules
3. The engine forces a **Strict JSON Chain-of-Thought (CoT)** schema. The LLM must output its response mapped perfectly to: $thought\_process$, $is\_medical$, $risk\_score$, $predicted\_specialization$, $confidence\_level$, and $emergency\_instructions$

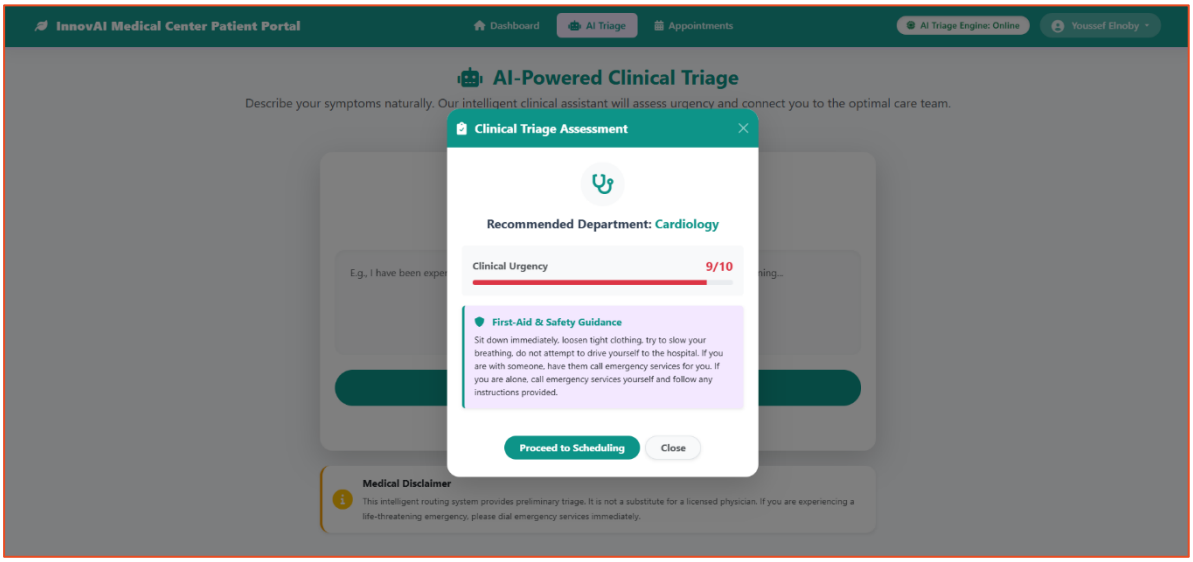

Figure 9 Output of the Llama 3 model enforcing the strict JSON schema, successfully mapping unstructured patient input to the dynamic Cardiology department array while returning critical First-Aid instructions.

4.2 Inbound Deep Search Algorithm & Payload Sanitization

For inbound WhatsApp webhook queries hitting $api/wa\_webhook.php$, the system must process unstructured text returned by the LLM. While the $llama-3.3-70b-versatile$ model is highly deterministic, LLMs occasionally inject markdown formatting wrappers (e.g., ```` ```json ... ``` ````) or conversational padding that can fatally break standard $json\_decode()$ functions

To completely immunize the database against these string formatting hallucinations, the system utilizes a custom **Regex Deep Search Algorithm**. Instead of attempting to parse the entire payload as a rigid object, the backend hunts for the exact keys using regular expressions, regardless of the surrounding syntax structure

Core Extraction Logic

```php
// Deep Search for Risk Score bypassing markdown wrappers
if (preg_match('/"[^"]*risk[^"]*"\s*:\s*"?([0-9]+(?:\/[0-9]+)?)"?/i', $ai_response, $matches)) {
    $risk_score = trim($matches[1]);
} else {
    $risk_score = "N/A"; // Fallback to prevent SQL type errors
}

// Deep Search for Predicted Specialization
if (preg_match('/"[^"]*specialization[^"]*"\s*:\s*"([^"]+)"/i', $ai_response, $matches)) {
    $predicted_spec = trim($matches[1]);
}
```

Figure 10 PHP code snippet illustrating the custom Regex Deep Search Algorithm used to safely extract Risk Score and Specialization data from the AI's response.

This algorithm ensures that only sanitized, strictly validated data variants are committed to the clinical ***AI_Analysis*** table, permanently preventing application-layer crashes caused by AI syntax drift

4.3 Asynchronous Execution Flow: AI Triage to WhatsApp Delivery

The following sequence diagram maps the exact asynchronous execution path mapping the patient UI, the Groq API, and the Node.js WebSocket gateway

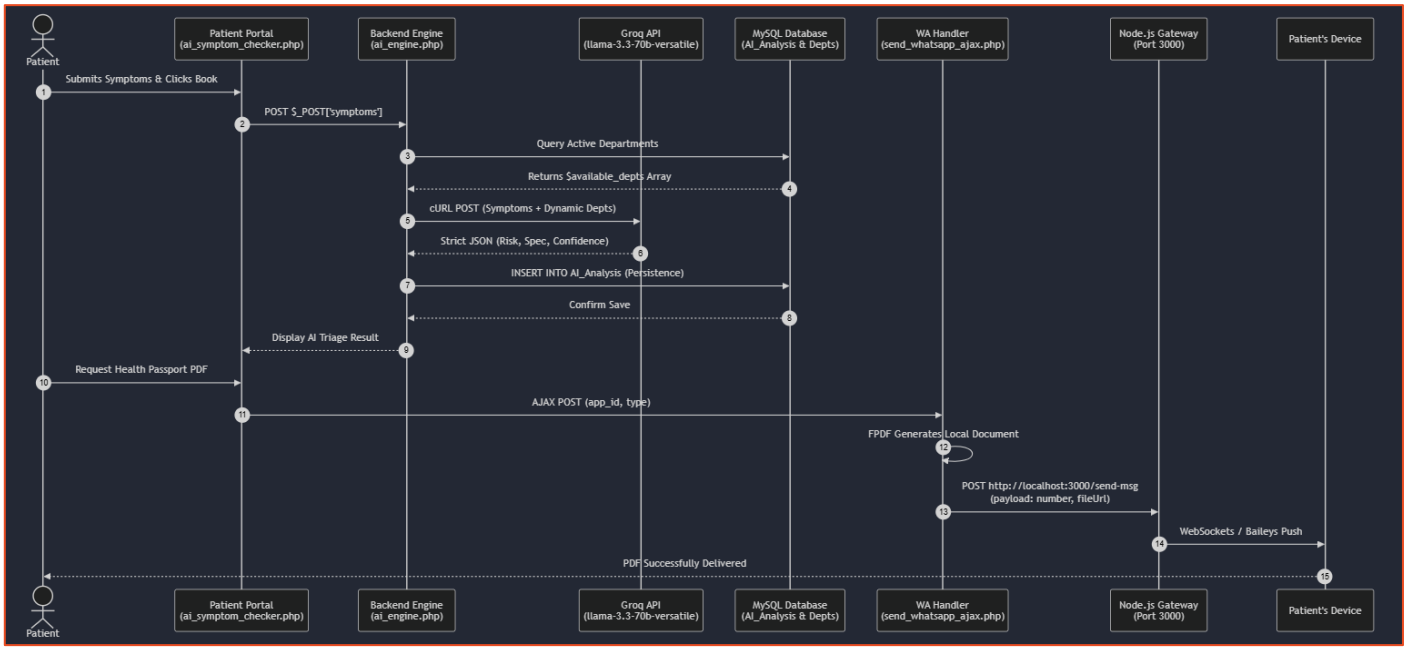

Figure 11 Sequence diagram detailing the asynchronous execution path across the decoupled architecture, mapping interactions between the frontend UI, PHP backend, Groq LLM API, MySQL database, and the Node.js WebSocket gateway.

5. Microservice & API Routing Map

To maintain a decoupled architecture, internal application logic is separated from external communication protocols. The **routing matrix** below details the primary payload handlers across the system:

Endpoint Context	Method	Payload / Parameters	Backend Processor	Action / Output
Baileys WA Microservice	***POST***	$JSON$: $\{number, message, fileUrl, fileType\}$	$localhost$: $3000/send-msg$	Relays asynchronous payloads directly to the **WhatsApp Web** protocol via $Node.js$
WhatsApp AI Webhook	***POST***	$JSON$: $\{number, message\}$	$api/wa\_webhook.php$	Hits the LLM; triggers $Node.js$ gateway if risk evaluates as High/Critical. Returns Text payload
Patient Document Delivery	***POST***	$Form\ Data$: $\{type, app\_id\}$	$send\_whatsapp\_ajax.php$	Generates **PDF** locally via **FPDF**, calls $\boldsymbol{Port\ 3000}$. Returns JSON status
Receptionist Patient Search	***GET***	$Query\ String$: $?q = [string]$	$search\_patients\_ajax.php$	Queried via $\boldsymbol{fetch()}$ in JS. Returns dynamic JSON array of matching Patient data
Receptionist QR Check-In	***POST***	$Form\ Data$: $\{appointment\_id\}$	$qr\_checkin\_ajax.php$	Instantly updates $\boldsymbol{Appointment.Status}$ to 'Checked-In'. Returns JSON success status

---

6. System Innovations: Security & UI/UX

6.1 Universal Gateway Security

Authentication is centralized through a unified $\boldsymbol{login.php}$ gateway. Plaintext passwords are mathematically salted and hashed utilizing PHP's native $\boldsymbol{password\_verify}$ function, enforcing robust $\boldsymbol{bcrypt/Argon2}$ cryptographic algorithms. Upon successful authentication, a dynamic switch ($\boldsymbol{SESSION['Role']}$) architecture strictly routes the session variable to the isolated, role-specific portal (e.g., isolating a user to their specific $Patient\_ID$ or $Admin\_ID$)

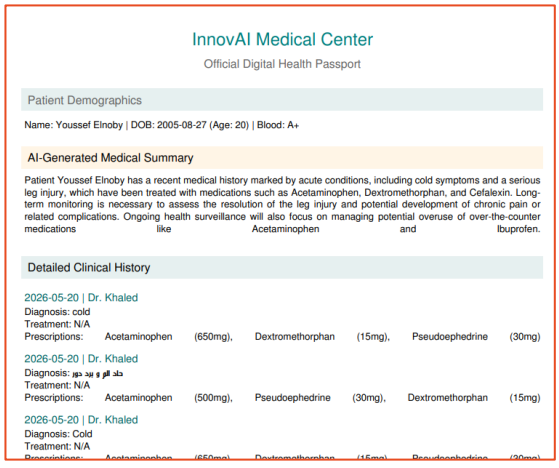

Figure 12 The final system deliverable: a comprehensive Digital Health Passport featuring an AI-generated medical summary and a complete clinical history.

6.2 Frontend Engineering

- **Edge Device Check-in (QR):** Driven natively by ***assets/js/qr_scanner.js*** using the Html5Qrcode library. It features a mirror-toggle webcam feed that automatically decodes the payload, pauses the UI stream, and fires a background POST request to instantaneously update the patient's status to Checked-In without requiring a page refresh
- **Omnipresent AJAX Search:** Data ingestion and retrieval are expedited via ***assets/js/ajax_handlers.js*** Querying ***search_patients_ajax.php*** beyond two characters renders dynamic DOM lists instantly, seamlessly binding specific database IDs to medical forms

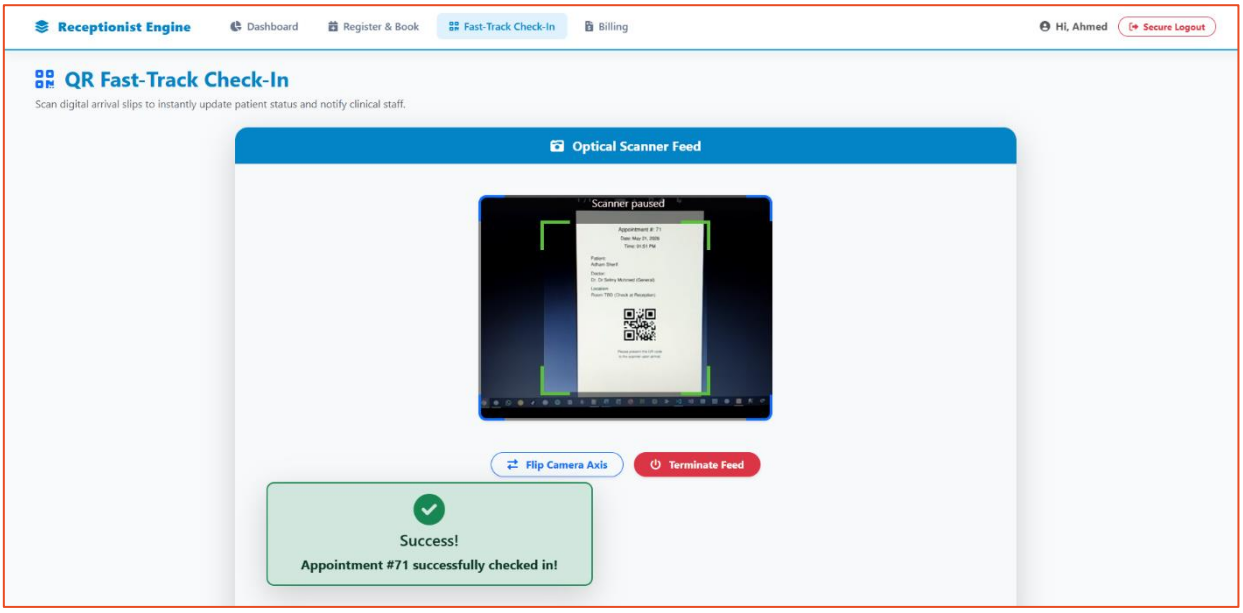

Figure 13 Frontend edge-device integration using the Html5Qrcode library. The script successfully decodes the payload and fires an asynchronous AJAX POST request to seamlessly update the database without blocking the UI thread.

Final Statement

This project served as a vital learning experience bridging full-stack software development, generative AI engineering, and enterprise network design. By combining mathematically rigorous database constraints with low-latency WebSocket microservices and deterministically constrained Large Language Models, we have *successfully engineered* a highly efficient, fault-tolerant backbone for modern healthcare administration

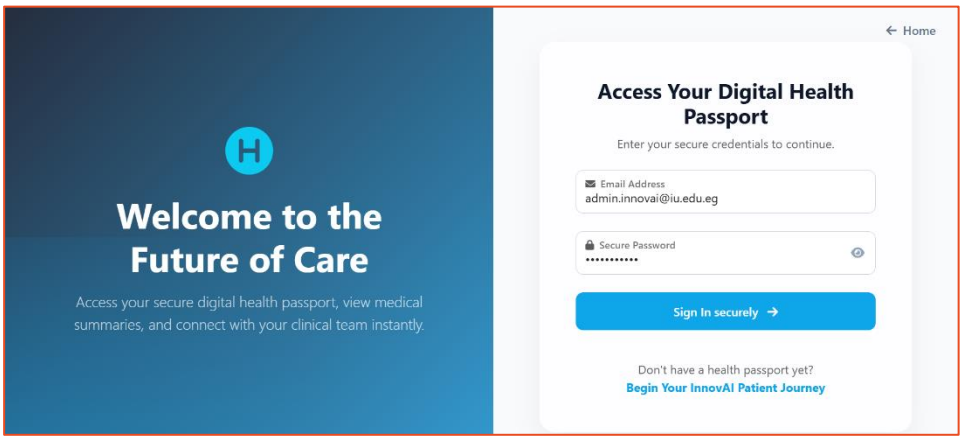

Figure 14 The unified authentication portal (login.php), which utilizes mathematically salted bcrypt hashing to secure credentials before executing role-based subnet routing.

GitHub Repository & Open-Source Codebase

The complete **project architecture**, including the source code, database schemas, and frontend assets, has been made ***open-source***. For a comprehensive review of the system, please access the *repository* via the ***URL*** below or by scanning the provided ***QR code***

https://github.com/yecreate/InnovAI-Hospital-Management-System